A Deep Reinforcement Learning Approach for Quadrotor Path Planning with Search-Based Planner Optimization

Xiaohui Chen[1], Yidong Chen[2], Li Liu[2], Hongbo Chen[1], and Yuhua Qi[1(✉)]

[1] School of Systems Science and Engineering, Sun Yat-Sen University, Guangzhou, China
qiyh8@mail.sysu.edu.cn

[2] China Academy of Launch Vehicle Technology, Beijing, China

Abstract. This paper proposes an end-to-end quadrotor path planning framework derived from deep reinforcement learning (DRL), which is sensor-based with Deep Deterministic Policy Gradient (DDPG) algorithm and search-based planner optimization. We use the current excellent search-based algorithm to demonstrate the path planning of the quadrotor to reach the goal and prioritize it into an experience buffer for replay. Semi-physical simulation experiments show that our framework can make the learning process converge quickly and realize the efficient use of information for exploration in the unknown environment, effectively improving the ability compare with the original DDPG algorithm in complex tasks.

Keywords: Reinforcement learning · Quadrotor · Path planning · Obstacle avoidance

1 Introduction

Quadrotors have a wide range of applications in agriculture, disaster relief, and wildlife observation and so on, becoming a popular research area [1–4]. To expand the scope of application scenarios and working capabilities of quadrotors, it is necessary to improve the autonomous navigation capabilities of quadrotors in unknown working environments.

The search-based path planning algorithm is a mature path planning algorithm. Based on the established search tree or graph, search strategies are used to find feasible or even optimal solutions. Dijkstra, A* and their improved algorithms such as D*, field D*, block A*, generalized adaptive A* and fringe saving A* [5,6] are efficient search-based algorithms. [7] suggested a Lyapunov-based coordination technique that enables A* algorithm-based low-level trajectory planning. [8] proposed bidirectional A* algorithms for path planning when measurements are intermittent. There is also a class of algorithms such as [9–13], which not only has a good path planning effect, but also optimizes the trajectory according to the flight characteristics of the quadrotor. Although search-based

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2023
L. Yan et al. (Eds.): ICGNC 2022, LNEE 845, pp. 6281–6290, 2023.
https://doi.org/10.1007/978-981-19-6613-2_606

algorithms have analytical completeness, when the map dimension is high or the size is large, it must process a substantial volume of data and information.

The unsupervised training of DRL and its interaction with the environment is well suited to the problem of quadrotor navigation in an unknown environment. [14] proposed a double deep Q-networks (DDQN) technique for maximizing data gathering while adhering to flight duration and navigation restrictions. In [15], Yan et al. solved the path planning problem by dueling double deep Q-networks (D3QN) algorithm with situational assessment, taking into account the probability of survival. However, when a quadrotor employs reinforcement learning to plan a course in an unfamiliar environment, it can only arrive at the optimal solution through trial and error. The applications of search-based path planning in quadrotor path planning have been mature with the characteristics of the quadrotor. Integrating search-based algorithms into reinforcement learning can not only make full use of known information to efficiently learn to accelerate convergence, but also dramatically improve the performance.

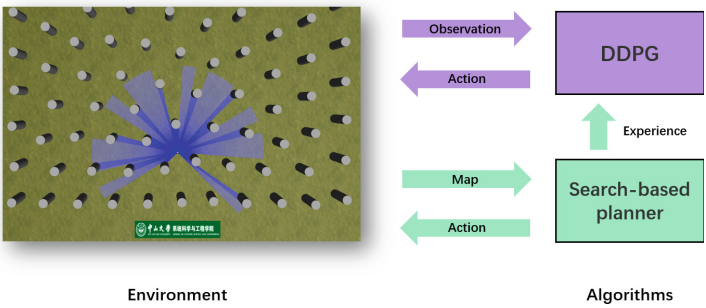


Fig. 1. An overview of sensor-based quadrotor reinforcement learning path planning and obstacle avoidance. Both algorithms are executed in the environment, with the search-based planner providing experience for reinforcement learning.

This paper proposes an end-to-end quadrotor path planning framework based on DRL combined with excellent search-based algorithms. Our framework enables the quadrotor equipped with lidar to learn path planning and obstacle avoidance in unknown environments (as shown in Fig. 1). These are the key contributions of this paper:

- An sensor-based effective framework is proposed that combines current effective search-based algorithms to enable the quadrotor to preferentially learn better strategies in a path planning environment.
- A DRL algorithm is designed that enables the quadrotor to achieve end-to-end path planning and obstacle avoidance in unknown environments using only laser perception.
- The framework is based on the real world and verified on a semi-physical simulation platform. Compared with original DDPG method, our framework has a faster convergence speed and better path planning and obstacle avoidance effect.

2 Problem Formulation

The path planning issue may be stated as the following Markov Decision Process (MDP) model: (S, A, P, R) [16], stand for state space (a description of environments), action space (a description of the quadrotor's behavior and the result of the quadrotor's decision.), state transition probability (a description of the conditional probability distribution of actions), and reward (a description of the environment's feedback to the quadrotor's actions), respectively. For path planning, during each t (time step), the quadrotor collects $s_t \in S$ and selects $a_t \in A$, with the state transition by $p(s_{t+1} \mid s_t, a_t)$ then $r(s_t, a_t)$ being provided. The value function Q is computed using the total of future rewards indicating the payout of the current action.

3 Approach

In this part, we discuss the reinforcement learning configuration, the specific characteristics of the DRL network that we created, and the process of the framework with an integrated implementation of the search-based method. See our framework diagram (Fig. 2) for details.

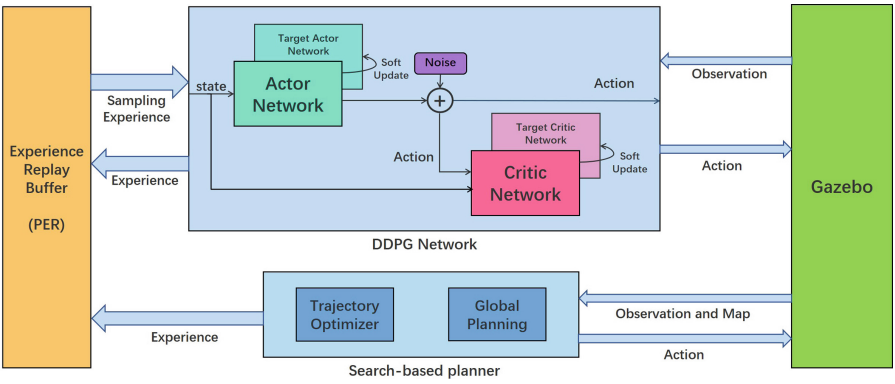


Fig. 2. The process of our framework: the search-based algorithm and DDPG are in the same environment for path planning. The search-based algorithm uses global planning and trajectory optimization to provide experience to the DDPG, and the DDPG uses an prioritized experience replay algorithm to extract the experience of both algorithms from the experience pool for training.

3.1 Reinforcement Learning Setup

Observation Space. The observation s_t consists of normalized laser data, quadrotor velocity, and relative goal position. The laser data is based on a real 2D lidar[1] with the range of 270° and 0.25° resolution. Specifically, the collected

[1] Hokuyo UST-10LX: https://www.hokuyo-aut.jp/.

raw laser data is sampled as 24 dimensions (as shown in Fig. 3) and normalized by its maximum value. The observed velocity includes the current translational and rotational velocity. The relative goal position is the distance and angle with the quadrotor.

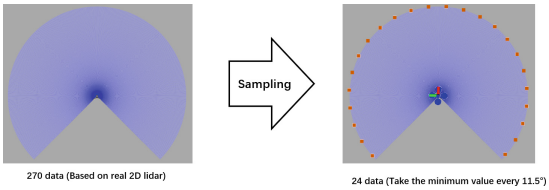


Fig. 3. For the sampling of laser data, the data volume changes from 1080 to 24, taking the minimum value every 11.25° and centering it.

Action Space. Since our environment is based on the real world, the flight control variables of the quadrotor are also continuously controlled. The quadrotor adopts a control mode that controls the translational velocity and rotational velocity $A \sim a_t\,[v_t, \omega_t]$. We selected the translational velocity range to $v \sim (0, 1)$ and the rotational velocity range to $\omega \sim (-1, 1)$ based on the kinematics of the quadrotor and real-world safety applications. The quadrotor is not allowed backward moving ($v \not< 0$) because the lidar could not cover the back area. For the real action output, we add random noise to ensure that the algorithm is exploratory.

Reward Design. We design the reward function from three aspects to avoid obstacles during reaching the goal and make the action smooth. The composition of reward r is:

$$r = r_g + r_c + r_s + r_a, \tag{1}$$

where r_g is reaching goal reward, r_c is the collision avoidance reward, r_s is action smoothing reward, r_a is action urge reward. The reaching goal reward guides the quadrotor to approach the goal:

$$r_g = \begin{cases} r_{arrived} & if\ \|p_t - p_g\| < d_{min} \\ \alpha\left(\|p_{t-1} - p_g\| - \|p_t - p_g\|\right) & \text{otherwise} \end{cases}, \tag{2}$$

where $r_{arrived}$ is the reward for get the goal, p_t is current position of the quadrotor, p_g is the position of goal, d_{min} is the distance to judge the arrival, α is a constant. Collision avoidance reward guides the quadrotor away from obstacles:

$$r_c = \begin{cases} 2r_{collision} & \text{if } \|p_t - O_k\| < R \\ r_{collision} & \text{if } R \leqslant \|p_t - O_k\| < 2R \\ r_{\text{laser}} & \text{otherwise} \end{cases}, \tag{3}$$

where $r_{collision}$ is collision warning reward, $O_k (0 \leq k \leq N)$ is the position of obstacle, R is collision distances, r_{laser} is laser distance reward. Action smoothing reward guides the quadrotor to fly smoother:

$$r_s = \begin{cases} r & \text{if } \|v_t - v_{t-1}\| > v_{min} \\ 0 & \text{otherwise} \end{cases} + \begin{cases} r_\omega & \text{if } \|\omega_t - \omega_{t-1}\| > \omega_{min} \\ 0 & \text{otherwise} \end{cases}, \tag{4}$$

where r_v is translational velocity smoothing reward, v_t is current translational velocity, v_{min} is translational velocity variation limitation, the term containing ω is the corresponding parameter of the translational velocity. Action urge reward prevents the quadrotor from stopping in place or flying too slowly:

$$r_a = \begin{cases} r_{uv} & \text{if } v_t < v_{umin} \\ 0 & \text{otherwise} \end{cases}, \tag{5}$$

where r_{uv} is translational velocity urge reward, v_{umin} is translational velocity limit.

We set $d_{min} = 0.25$, $r_{\text{arrived}} = 100$, $\alpha = 420$, $R = 0.25$, $r_{\text{collison}} = -100$, $r_v = r_\omega = -2$, $v_{min} = \omega_{min} = 0.5$, $r_{uv} = -2$, $v_{\text{umin}} = 0.2$ in the training.

3.2 DRL Network

Actor network maps observation data to actions (in Fig. 4(a)). We develop a five-hidden-layer neural network to approximate the nonlinear function π. The first three layers are fully connected networks with 500 rectifier units and ReLU activation functions. Then the last two full connected layers output the rotational velocity through the Tanh activation function with range $(-1, 1)$, and the normalized value $(0, 1)$ of the translational velocity through the Sigmoid activation function. Critic network maps observation data and actions to Q-values (in Fig. 4(b)). After the observation data passes through a fully connected layer, it is merged with the current action. After two fully connected layers, the Q value is output through linear activation.

3.3 Search-Based Optimization

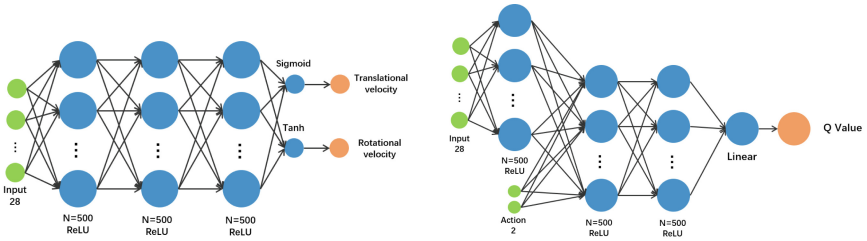
To take advantage of the search-based method, we use it as a demonstration to put data into an experience pool in the same format as reinforcement learning approaches. To enhance the utilization of effective experience, we add a Prioritized Experience Replay (PER) [17] mechanism which enables efficient propagation of reward information.

The priority is set by four parameters, including the last TD error δ, dominance constant ε_d, equilibrium constant ε, and network loss $J(\theta)$ which can be expressed as $J(\theta) = \nabla_a Q\left(s_i, a_i \mid \theta^Q\right)$ (weighted by λ):

$$p_i = \varepsilon_d + \varepsilon + \delta_i^2 + \lambda |J(\theta)|^2. \tag{6}$$

Then, the probability that the experience is sampled can be computed as:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}, \tag{7}$$



(a) Actor Network(Output action from state)

(b) Critic Network (Output value function from state and action)

Fig. 4. Actor-Critics Network: (a) is the actor network with 5 fully connected layers, with the state as input and action instructions as output. (b) is a critic network with 4 fully connected layers. State and action are inputs and Q value is output.

where $P(i)$ represents the sampling probability, p_i represents the priority, α is the distribution factor. To account for the change in sampling probability, update the network with the following weights:

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)}\right)^{\beta}, \tag{8}$$

where N denotes batch size, β is a constant to adjust sampling weights. We set $\varepsilon_d = 0.4$, $\varepsilon = 0.2$, $\lambda = 0.2$, $\alpha = 0.3$, $\beta = 1$ in the training.

Remark 1. We select EGO-Planner [10] as a search-based method. In EGO-Planner, the collision-free path is searched by A* method and optimized by the gradient-based spline optimizer using smoothness, collision, and dynamic feasibility terms. We use globally plan by a global point cloud and optimize trajectories, implementing EGO-Planer onto our platform. We only collect observation states and actions for reinforcement learning during the planning process, then takes all data into the experience buffer for priority replay.

4 Experiments and Results

In this part, we introduce our platform, display and evaluate training results, and show the experimental results in the test.

4.1 Experiments Setup

Our experiments are conducted on a PC with a 7th Intel Core CPU (i7-8700K) and a GPU (Nvidia GTX 1650). The experiments run on the Ubuntu system, while the robot-related components make use of the ROS and the machine learning-related components make use of the Keras based on TensorFlow. The experiments take place in Gazebo, a semi-physical simulation environment.

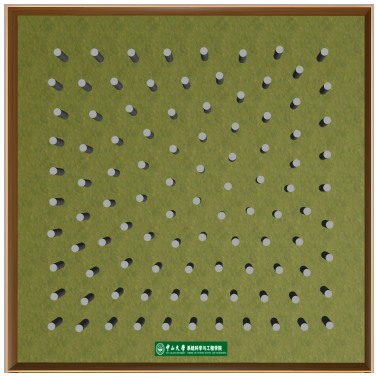


Fig. 5. The path planning environment for the quadrotor. The number of obstacles is 100, and the occupied area is 20 m * 20 m.

To minimize the effort to transfer the approach from simulation to real experiment, we use an open-source autonomous drone software platform Prometheus[2], which supports simulation and experiment for quadrotors, to interface with the Gazebo environment. For environments during training, we generate obstacles using the K-means approximation (in Fig. 5) to create training scenarios.

4.2 Training Evaluation

We compare with the original DDPG method. The two methods run on the same platform, and the training iterations is 800, the learning rate is 0.0001, the batch size is 512. The comparisons are detailed in Table 1.

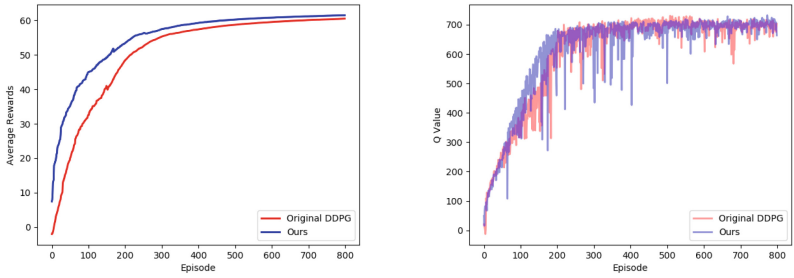
Table 1. Training settings

Method	Experience buffer	Episode	Time consuming
Original DDPG	Only DDPG	800	300 min
Ours	Ego and DDPG	800	254 min

We train the strategies in the same way, and the training effects is shown in Fig. 6. The evaluation indicators include the average reward and Q value of each episode. It can be seen that our method has a good initial reward close to 10 at the beginning of training, while the normal initial reward level is around 0. Our method tends to converge at 200 iterations, while the original DDPG only converges at 300 iterations, showing that adding valid empirical data can greatly accelerate network convergence. It can also be seen from the Q value that our method will also obtain a better Q value as soon as possible. In all

[2] https://github.com/amov-lab/Prometheus.

training processes, our method outperforms the original DDPG at the reward level throughout the training process. Our method finally stabilizes at 63.7, and the original DDPG only stabilizes at 60.6.



(a) The average rewards of frameworks (b) The Q value of frameworks

Fig. 6. Average rewards and Q value for the training. Our method beats the original DDPG greatly in training time and somewhat in the ultimate reward.

4.3 Testing Evaluation

We test the success rate of path planning for the methods. In test environment, the starting position is randomly selected from the beginning (20 m from the same horizontal line), and the goal is randomly selected from the end (20 m from the same horizontal line). The path planning distance is greater than or equal to 20 m. We test the methods 500 times and compute the total success rate.

Table 2. Implementation comparison of the methods

Method	Episode	Success rate	Average time
Original DDPG	500	31.91%	23.45 s
Ego optimization	500	97.12%	23.32 s

In Table 2, The original DDPG has just a 31.91% success rate, and its obstacle avoidance ability in such a long-distance environment with dense obstacles is limited. Our method can achieve a success rate of 97.12% in such a dense and long-distance environment, and has a excellent path planning and obstacle avoidance effect. Our success rate is 204.36% higher than the original DDPG, with much higher performance than original DDPG. We randomly selected four sets of velocity in the test to demonstrate the control process of our method on the quadrotor in Fig. 7.

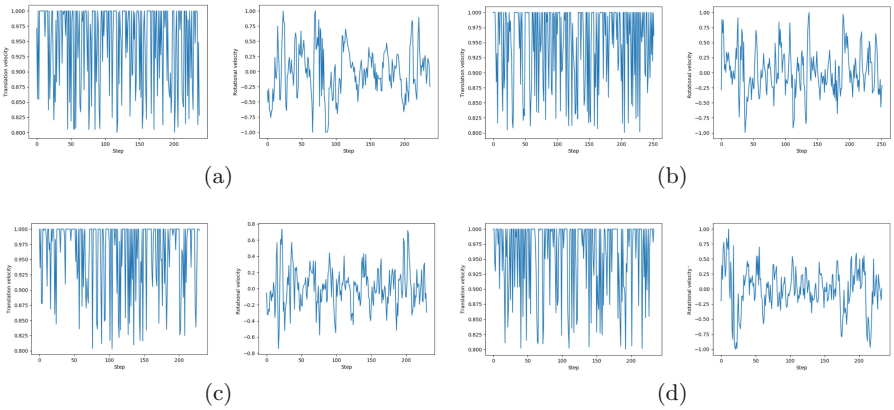


Fig. 7. Four sets of translational velocity and rotational velocity distribution of successful path planning in the test, which is the output value of the actor network.

5 Conclusion

We propose a quadrotor end-to-end DRL framework incorporating a search-based approach, demonstrated using the EGO-planner and employing prioritized experience replaying, in this paper. Experiments demonstrate that the framework is effective at increasing training speed and is effective at solving the quadrotor path planning problem. In future work, we intend to incorporate 3D radar, depth map, and other 3D sensing data to enable path planning in the quadrotor's 3D environment, and eventually to carry it to a physical object.

References

1. Qi, Y., Zhu, Y., Wang, J., Shan, J., Liu, H.H.: MUDE-based control of quadrotor for accurate attitude tracking. Control Eng. Prac. **108**, 104,721 (2021)
2. Wang, Z., Zhou, X., Xu, C., Gao, F.: Geometrically constrained trajectory optimization for multicopters. arXiv preprint arXiv:2103.00190 (2021)
3. Chen, T., Shan, J., Liu, H.H.: Cooperative transportation of a flexible payload using two quadrotors. J. Guid. Control. Dyn. **44**(11), 2099–2107 (2021)
4. Qi, Y., Jiang, J., Wu, J., Wang, J., Wang, C., Shan, J.: Autonomous landing solution of low-cost quadrotor on a moving platform. Robot. Auton. Syst. **119**, 64–76 (2019)
5. Rios, L.H.O., Chaimowicz, L.: A survey and classification of a* based best-first heuristic search algorithms. In: da Rocha Costa, A.C., Vicari, R.M., Tonidandel, F. (eds.) SBIA 2010. LNCS (LNAI), vol. 6404, pp. 253–262. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-16138-4_26
6. Hyla, P., Szpytko, J.: Automated guided vehicles: the survey. J. KONES **24** (2017)
7. Ma, X., Jiao, Z., Wang, Z., Panagou, D.: 3-d decentralized prioritized motion planning and coordination for high-density operations of micro aerial vehicles. IEEE Trans. Control Syst. Technol. **26**(3), 939–953 (2017)

8. Penin, B., Giordano, P.R., Chaumette, F.: Minimum-time trajectory planning under intermittent measurements. IEEE Rob. Autom. Lett. **4**(1), 153–160 (2019). https://doi.org/10.1109/LRA.2018.2883375
9. Zhou, B., Pan, J., Gao, F., Shen, S.: Raptor: Robust and perception-aware trajectory replanning for quadrotor fast flight. IEEE Trans. Rob. **37**(6), 1992–2009 (2021)
10. Zhou, X., Wang, Z., Ye, H., Xu, C., Gao, F.: Ego-planner: an esdf-free gradient-based local planner for quadrotors. IEEE Rob. Autom. Lett. **6**(2), 478–485 (2020)
11. Ye, H., Zhou, X., Wang, Z., Xu, C., Chu, J., Gao, F.: TGK-planner: an efficient topology guided kinodynamic planner for autonomous quadrotors. IEEE Rob. Autom. Lett. **6**(2), 494–501 (2020)
12. Zhou, B., Gao, F., Wang, L., Liu, C., Shen, S.: Robust and efficient quadrotor trajectory generation for fast autonomous flight. IEEE Rob. Autom. Lett. **4**(4), 3529–3536 (2019)
13. Zhou, X., Zhu, J., Zhou, H., Xu, C., Gao, F.: Ego-swarm: a fully autonomous and decentralized quadrotor swarm system in cluttered environments. In: 2021 IEEE International Conference on Robotics and Automation (ICRA,, pp. 4101–4107. IEEE (2021)
14. Bayerlein, H., Theile, M., Caccamo, M., Gesbert, D.: Uav path planning for wireless data harvesting: a deep reinforcement learning approach. In: GLOBECOM 2020-2020 IEEE Global Communications Conference, pp. 1–6. IEEE (2020)
15. Yan, C., Xiang, X., Wang, C.: Towards real-time path planning through deep reinforcement learning for a UAV in dynamic environments. J. Intell. Rob. Syst. **98**(2), 297–309 (2020)
16. Lillicrap, T.P., et al.: Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971 (2015)
17. Schaul, T., Quan, J., Antonoglou, I., Silver, D.: Prioritized experience replay. arXiv preprint arXiv:1511.05952 (2015)